

Java Programming 3

Week 8 - Relations



Agenda this week

Project review

Implementing relationships in the repository

One-To-One

Many-To-One

Many-To-Many



Agenda this week

Project review
Implementing relationships in the repository
One-To-One
Many-To-One
Many-To-Many

Project Review

KdG





Agenda this week

Project review

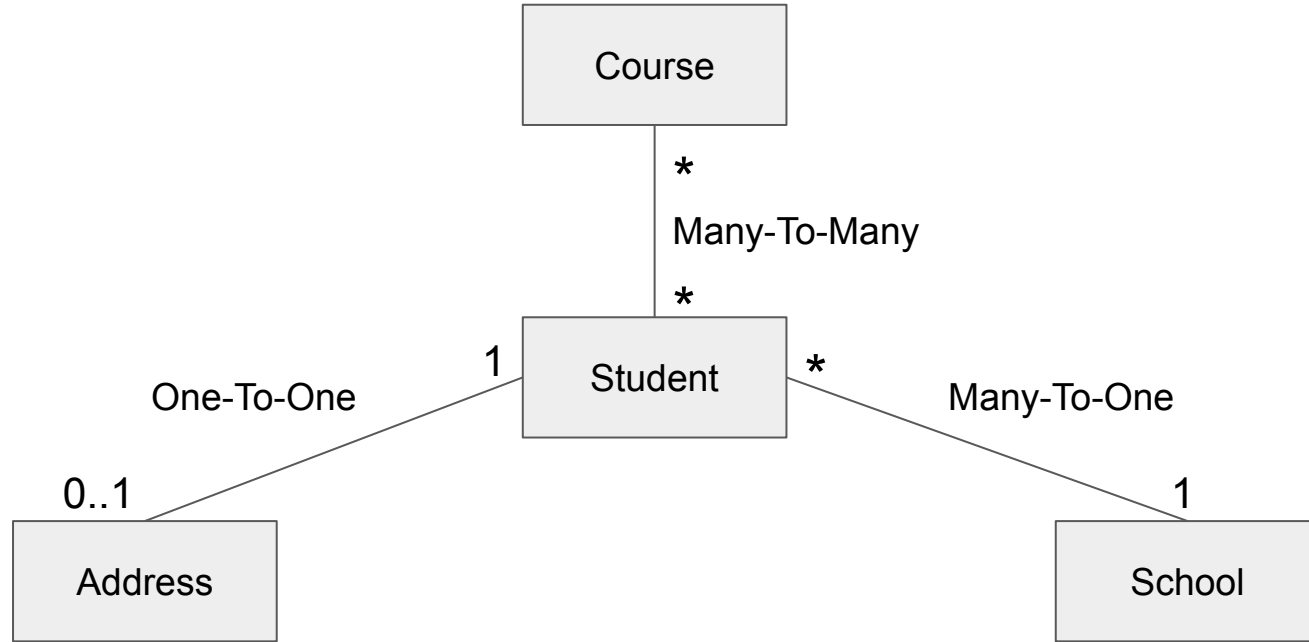
Implementing relationships in the repository

One-To-One

Many-To-One

Many-To-Many

Relationships in the repository





Agenda this week

Project review

Implementing relationships in the repository

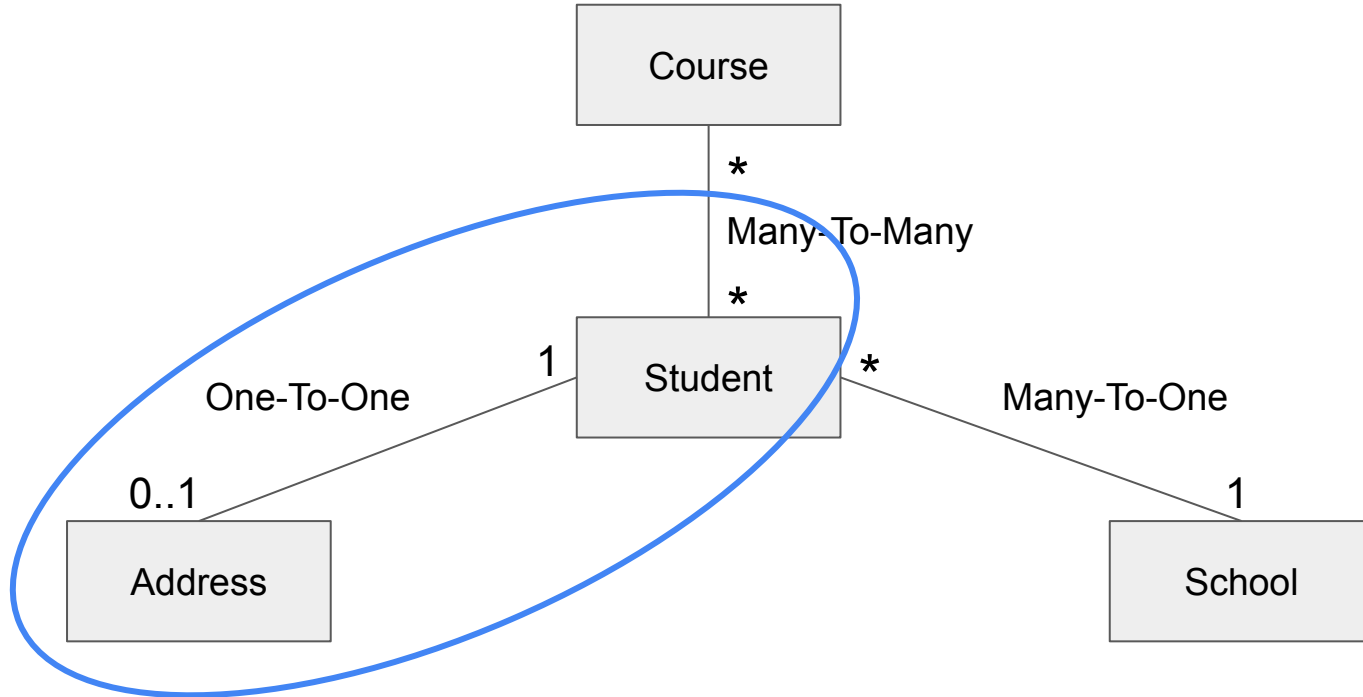
One-To-One

Many-To-One

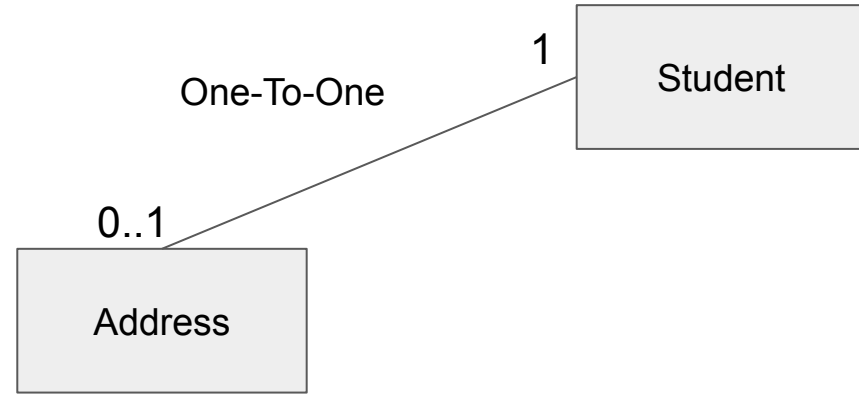
Many-To-Many

Relationships in the repository

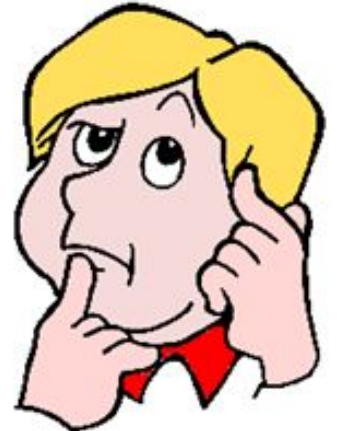
- Student versus Address



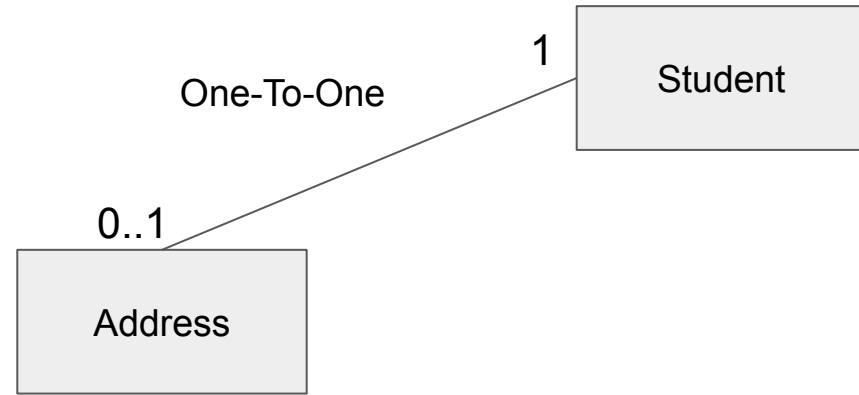
One-To-One



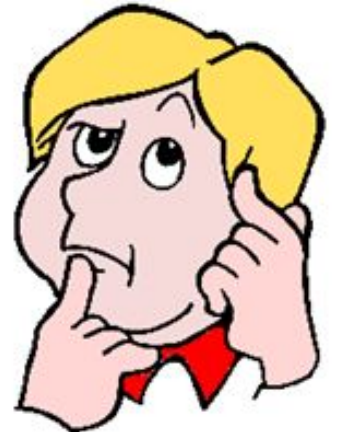
- Student has 1 Address
- Each Address belongs to 1 Student
- The Address is not necessary: a Student can exist without an Address



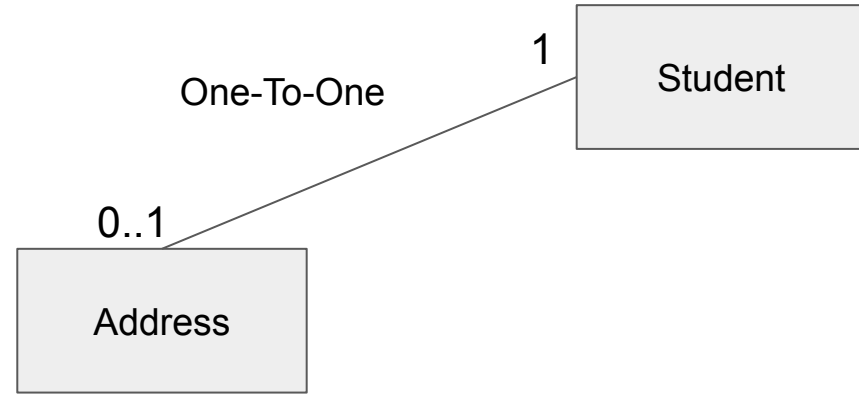
One-To-One: in Database



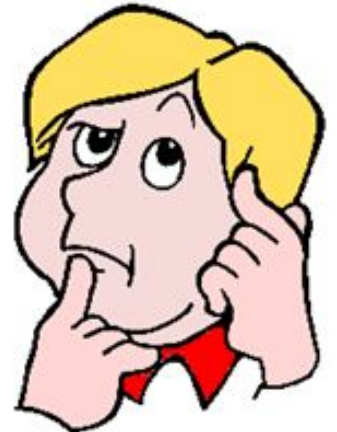
- Different options...
 - Table STUDENT and Table ADDRESS?
 - STUDENT has foreign key to ADDRESS?
 - ADDRESS has foreign key to STUDENT?
 - Separate STUDENT_ADDRESS table with STUDENT_ID and ADDRESS_ID?
 - ...?
 - Table STUDENT and extra columns for Address information? (“embedded”)
 - ...?



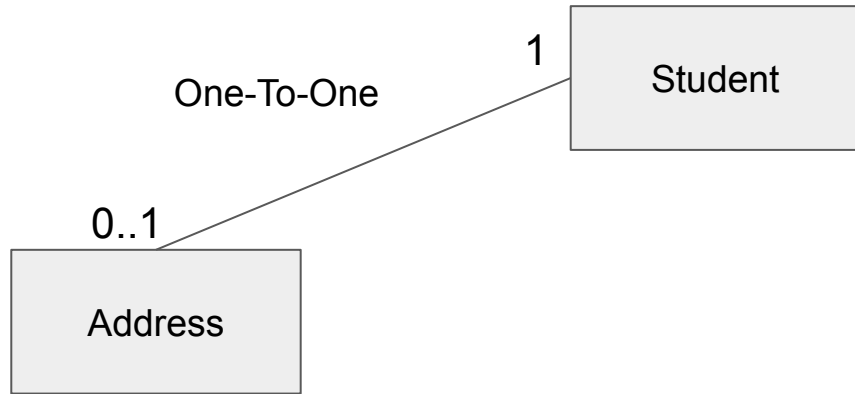
One-To-One: in Domain Model



- Student and Address class
- Different options...
 - Unidirectional:
 - From Student to Address:
 - Student had Address attribute
 - Address does not have a Student attribute
 - From Address to Student
 - Address has Student attribute
 - Student does not have an Address attribute
 - Bidirectional
 - Student has Address attribute
 - Address has Student attribute



One-To-One: we choose...



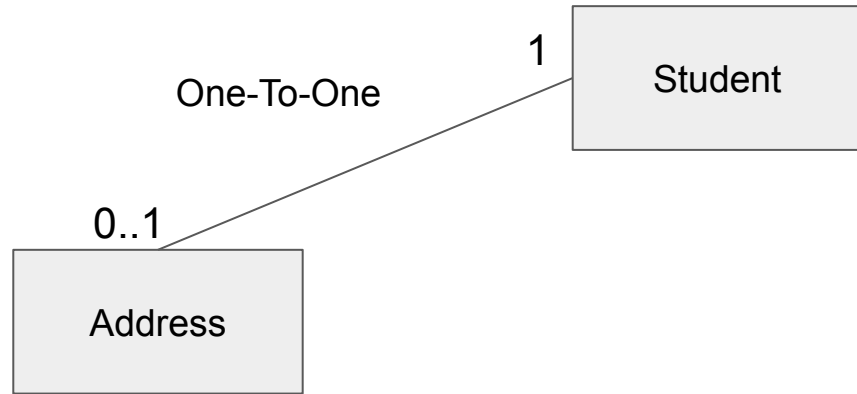
- Database:

- Embedded: no separate ADDRESS table
- The ADDRESS columns are added to the STUDENT table

```
create table STUDENTS
(
  ID          INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  NAME        CHARACTER VARYING(100),
  LENGTH      DOUBLE PRECISION,
  BIRTHDAY    DATE,
  STREET       CHARACTER VARYING(100),
  POSTAL_CODE INTEGER,
  CITY        CHARACTER VARYING(100)
);
```



One-To-One: we choose...



- Domain Model:

- Class Address (and of course class Student)
 - No id field: *Address is not an entity* → *has no meaning on it's own...*
- Unidirectional:
 - Student has Address attribute (with setter and getter)
 - Address has NO Student attribute

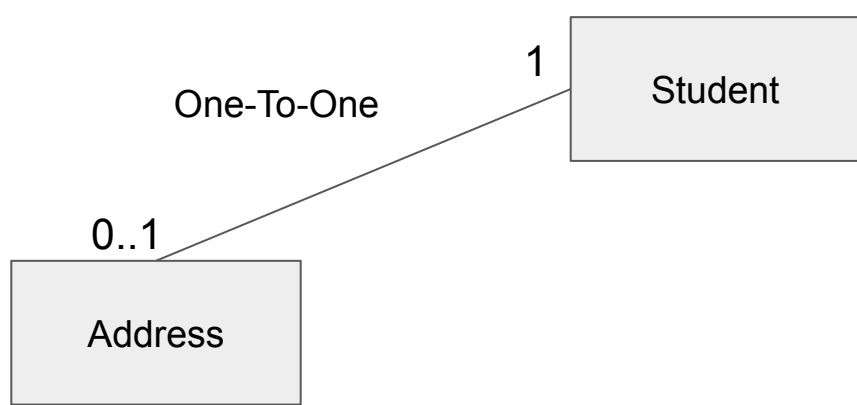
```
public class Student {
    private int id;
    private String name;
    private double lenght;
    private LocalDate birthday;
```

```
//relation: One to One
private Address address;
```

```
public class Address {
    private String street;
    private int postalCode;
    private String city;
```



One-To-One: let's do it!



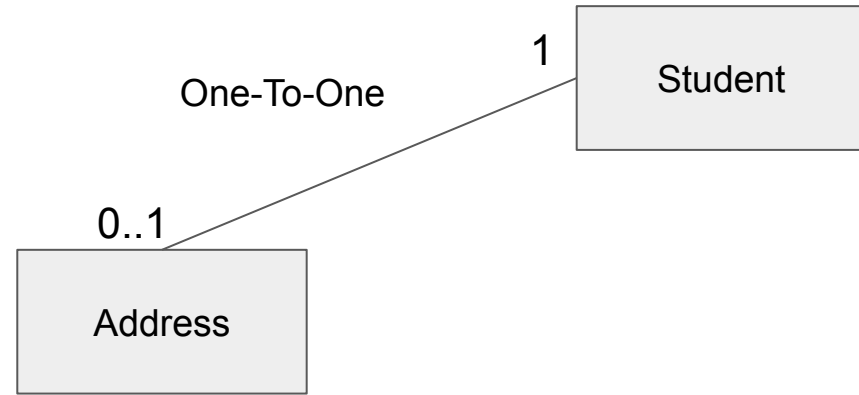
- Start from this code:
<https://gitlab.com/kdg-ti/programming-3/exercises/relationsdemo>
- Database:
 - Create an H2 file database based on the schema.sql and data.sql
 - Add the address columns to STUDENT
 - Generate the DDL and add to the schema.sql
 - Add some addresses to students, generate SQL INSERTS and add to data.sql
 - Run the application on the h2 memory database
 - Check <http://localhost:8080/h2-console>
 - available, because we have spring-boot-starter-web and in application.properties we have

`spring.h2.console.enabled=true`

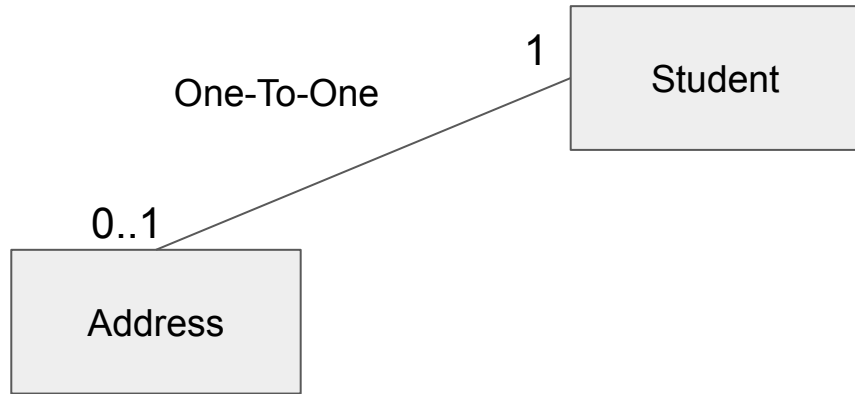


One-To-One: let's do it!

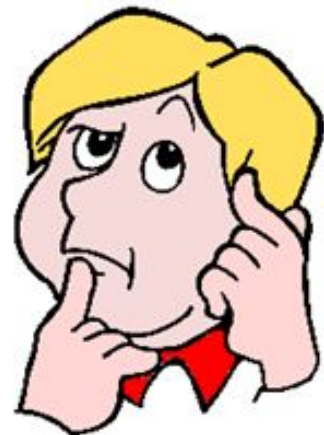
- Domain Model:
 - Create Address class in domain package
 - No id attribute
 - street, postalCode, city attribute
 - Getters, setters, toString
 - Student class:
 - Add Address attribute to Student class
 - Getter and setter
 - Add address to the toString



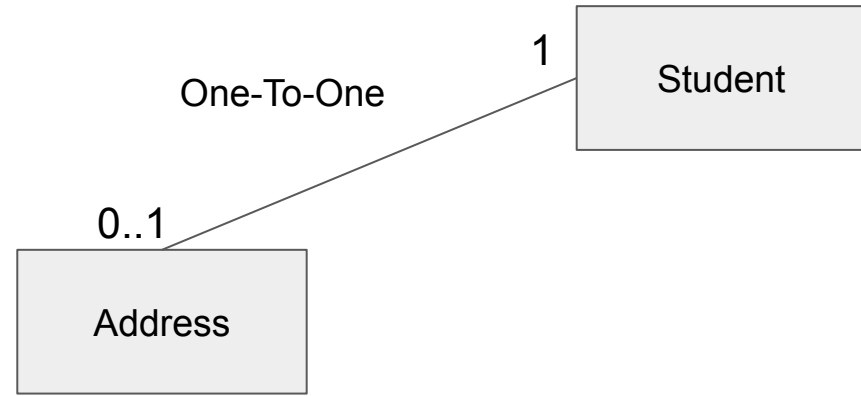
One-To-One: the repository?



- **findAll - findById methods**
 - When we load the Student, should we load the Address?
 - Eager loading: yes we do
 - Lazy loading: no we don't
 - For One-To-One: eager loading often good idea... → let's do it like this!
- **createStudent method**
 - If the Student has an Address, we insert it into the address columns
- **updateStudent method**
 - If the Student has an Address, we insert or update the address columns
- **deleteStudent method**
 - No changes



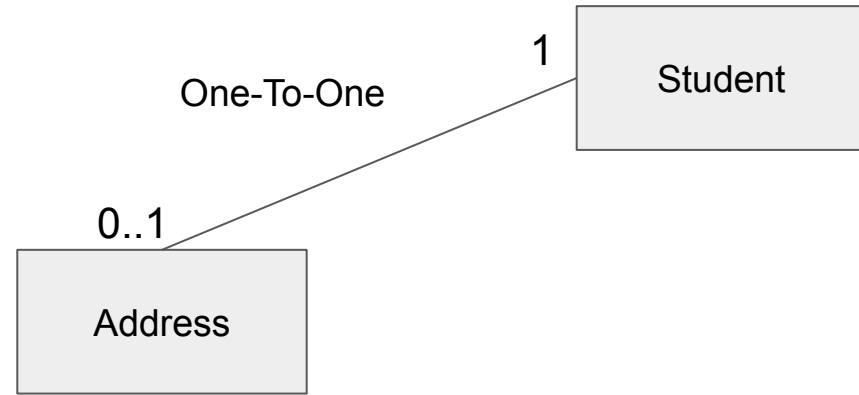
One-To-One: let's do it



- Implement the repository methods:
 - findAll - findById methods → eager loading of Address
 - createStudent method
 - updateStudent method
 - deleteStudent method



One-To-One: StudentMenu



- StudentMenu:
 - Test the “list all students”
 - Should show the Address if it exists
 - Implement the “change address of student”
 - You ask for student id
 - You ask for Street, Postal code and City
 - Create and set the Address of the Student
 - Call updateStudent method on repository





Agenda this week

Project review

Implementing relationships in the repository

One-To-One

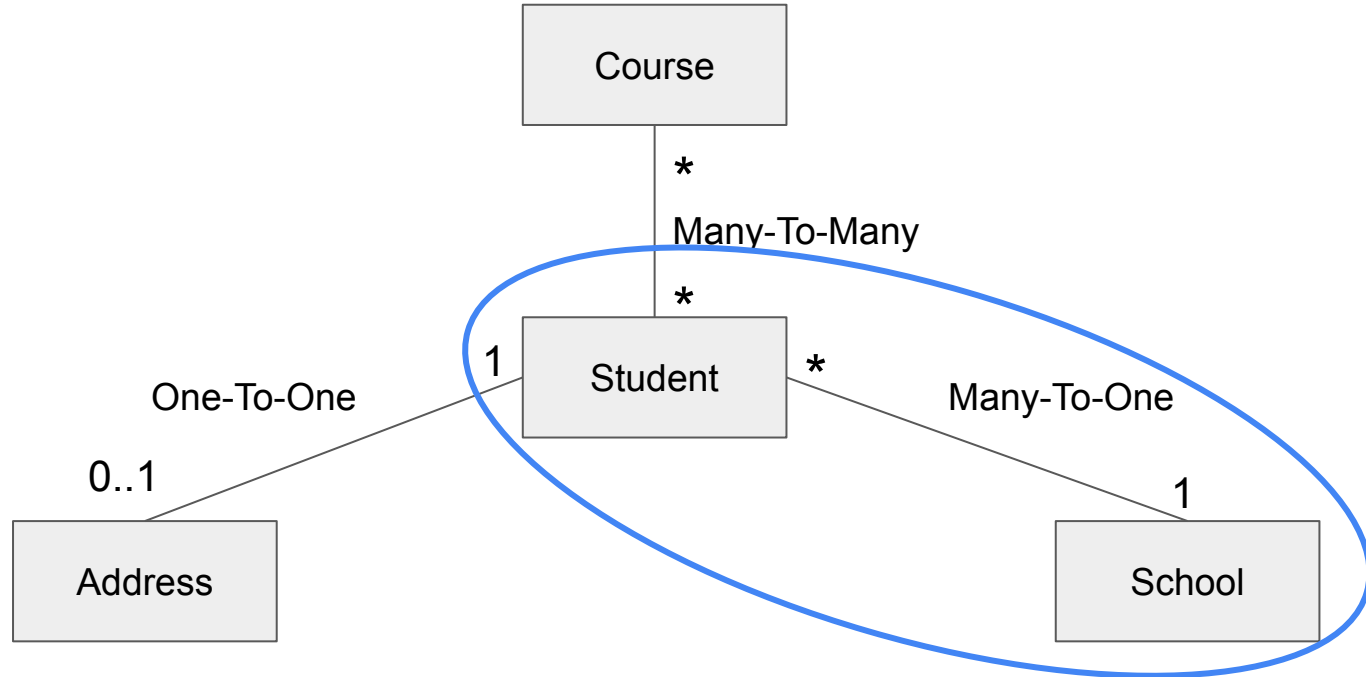
Many-To-One

Many-To-Many

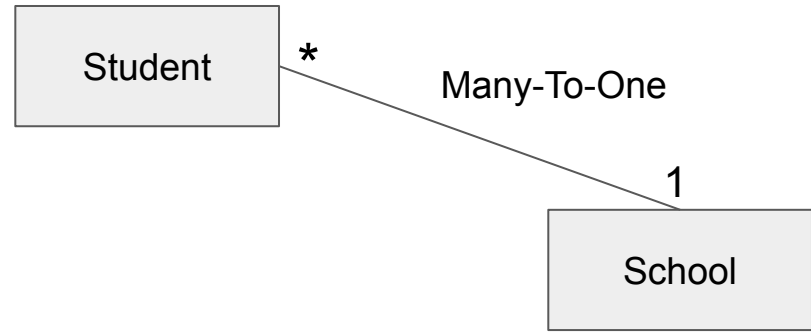
Generics: implementing a generic repository → this part will be covered later

Relationships in the repository

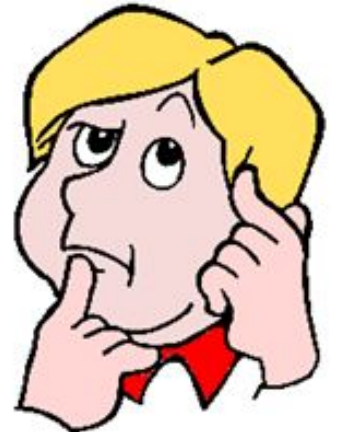
- Student versus School:



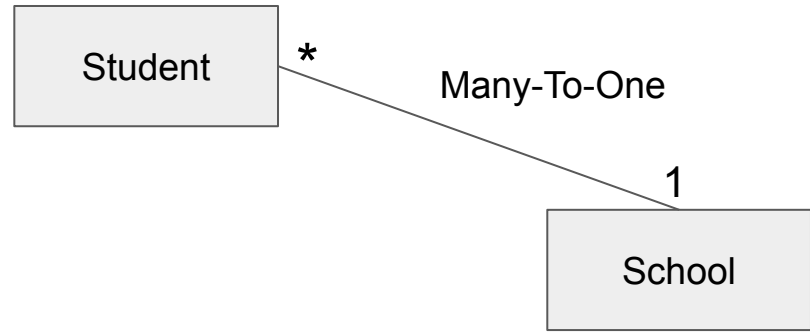
Many-To-One



- Student has 1 School
- School has many Students
- A Student always has a School, a Student without a School cannot exist



Many-To-One: Database

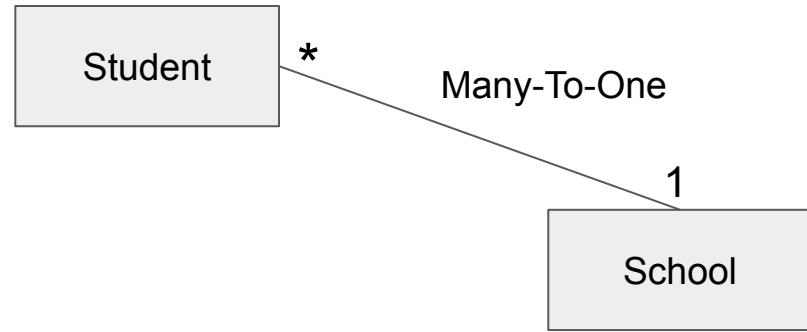


- Table SCHOOLS
 - School has an ID and a NAME → School is an entity
- Table STUDENTS
 - Has a SCHOOL_ID: foreign key to SCHOOLS

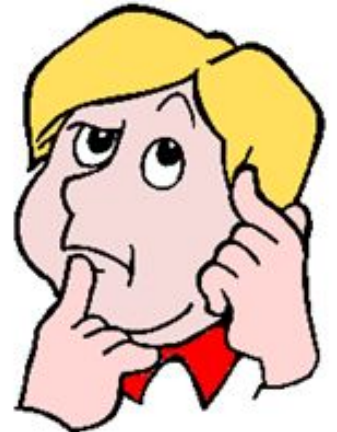
```
create table SCHOOLS
(
  ID      INTEGER auto_increment
          primary key
          unique,
  NAME CHARACTER VARYING(100) not null
);
```

```
create table STUDENTS
(
  ...
  SCHOOL ID INTEGER not null,
  constraint FK SCHOOL ID
          foreign key (SCHOOL_ID) references SCHOOLS
);
```

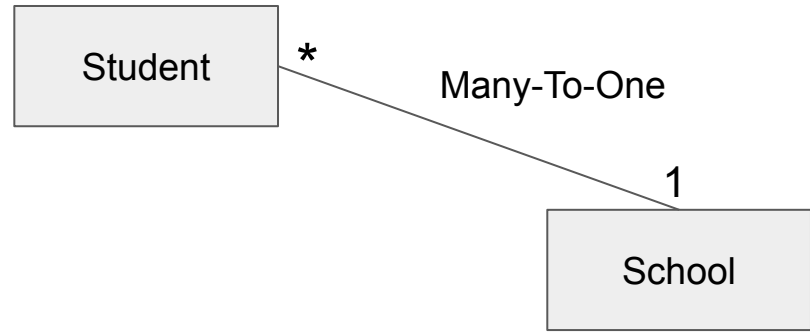
Many-To-One: in Domain Model



- **School class**
 - Has a `List<Student>` attribute and an `addStudent` method
 - `toString` method does not show the students
- **Student class**
 - Has a `School` attribute with getter and setter
 - Setter adds student to the `School` using `addStudent` method
 - `toString` method shows the `School`



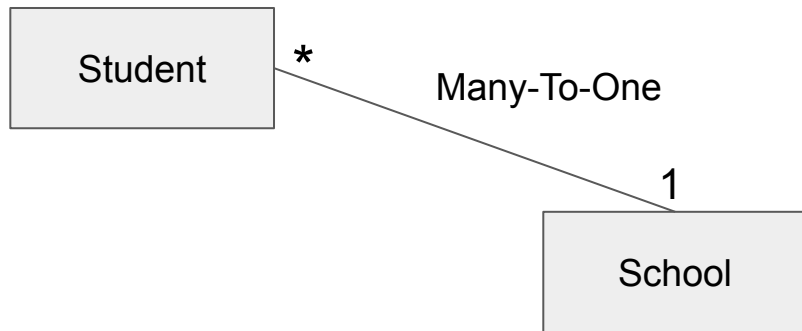
Many-To-One: let's do it!



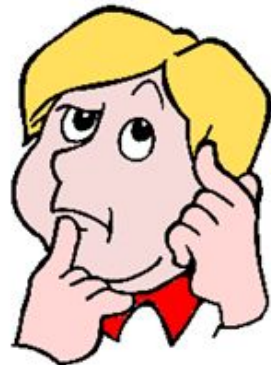
- Database:
 - Add SCHOOLS table to database
 - Alter the STUDENTS table: add SCHOOL_ID with FK to SCHOOLS
 - Add some data
 - Generate DDL and SQL INSERTS, add to schema.sql and data.sql
- Domain Model
 - Create School class, has id and name. Create constructors, getters, setters, toString
 - Add School to Student, List<Student> to School
 - Update toString of Student



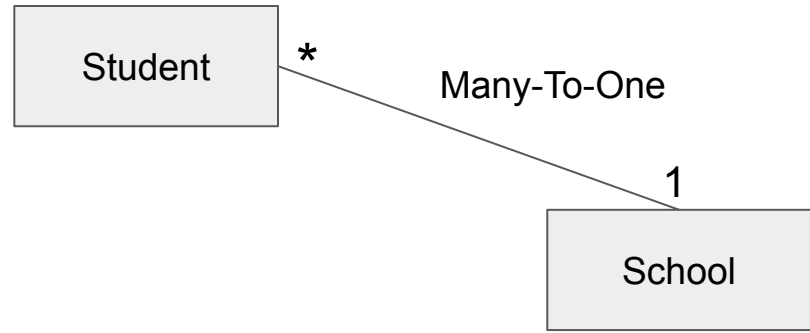
Many-To-One: repository?



- We add a SchoolRepository
 - Interface - implementation
 - findAll - findById
 - Do we load the students → let's not do that!
 - createSchool - updateSchool
 - deleteSchool:
 - If we delete a school, should we delete the students?
 - Yes: students without a school cannot exist → Use @Transactional!
- In StudentRepository
 - findAll en findById: should we load the School?
 - Many-to-One: ok, let's do it
 - Let's add a findBySchool method
 - createStudent → add the SCHOOL_ID
 - School should already exist, we don't create it here
 - updateStudent → add the SCHOOL_ID



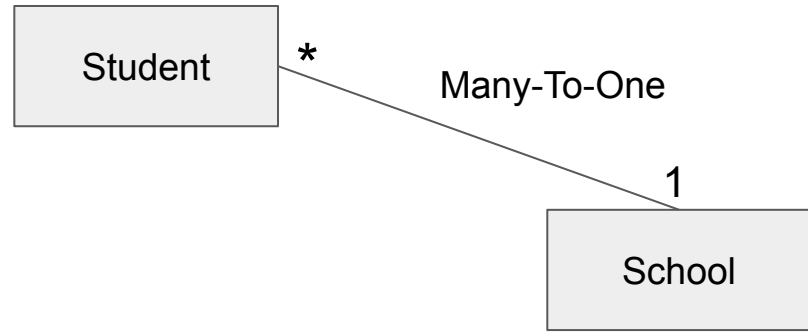
Many-To-One: let's do it



- We add a SchoolRepository
 - Interface - implementation
 - findAll - findById → do not load the students
 - createSchool - updateSchool
 - deleteSchool → delete the students → delete the students addresses
- In StudentRepository
 - findAll en findById: → load the School
 - Let's add a findBySchool method
 - createStudent, updateStudent → add the SCHOOL_ID



Many-To-One: StudentMenu



- **StudentMenu:**

- Test the “list all students”: should show school information
- `addStudent`
 - Ask for to (E)xisting or (N)ew school
 - New → ask school name and create school using `schoolRepository`
 - Existing → ask school id and `findById` from `schoolRepository` and `setSchool` on student object
- `updateStudent`: idem!
- Implement “Change school of student” → to only change school
- Implement “list all students of school” → use `findBySchool` method of `studentrepo`
- Implement “Delete school” → use `schoolRepository`





Agenda this week

Project review

Implementing relationships in the repository

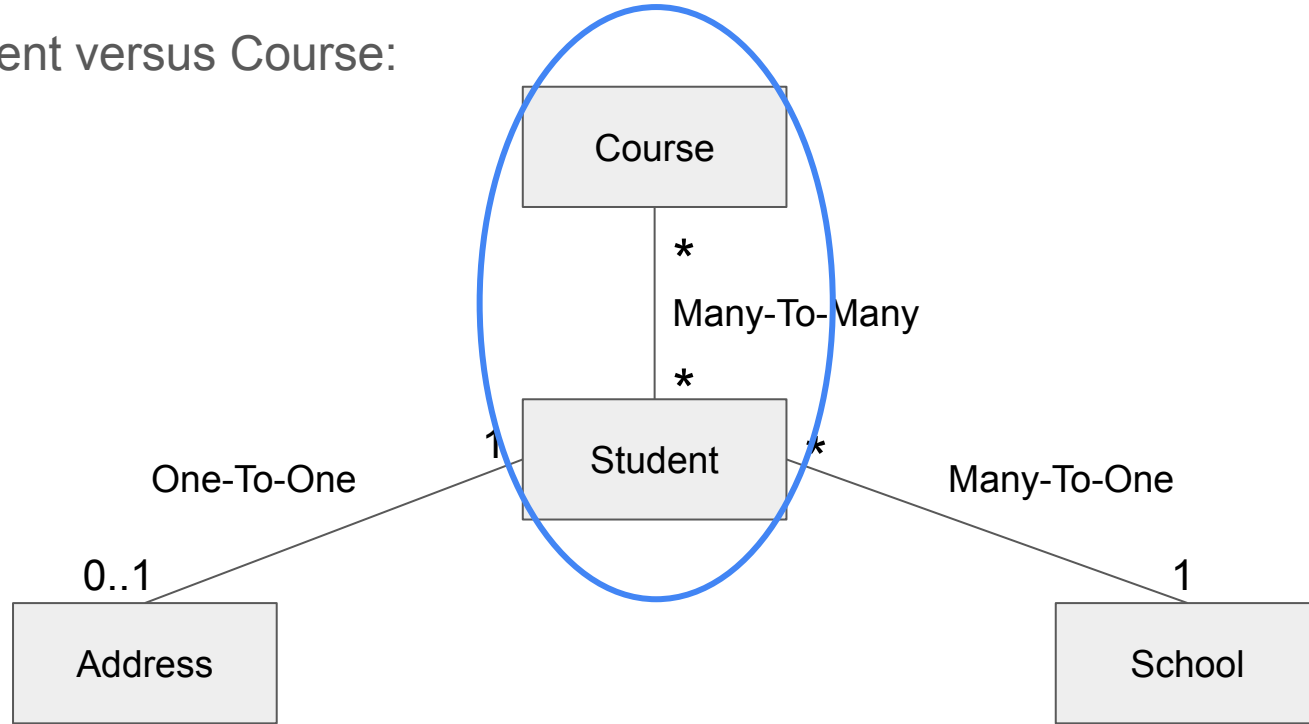
One-To-One

Many-To-One

Many-To-Many

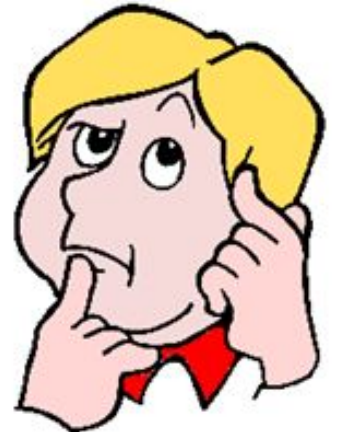
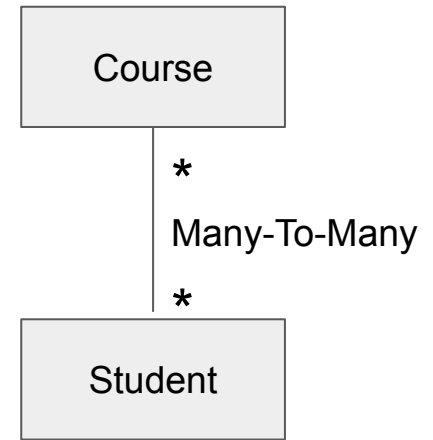
Relationships in the repository

- Student versus Course:



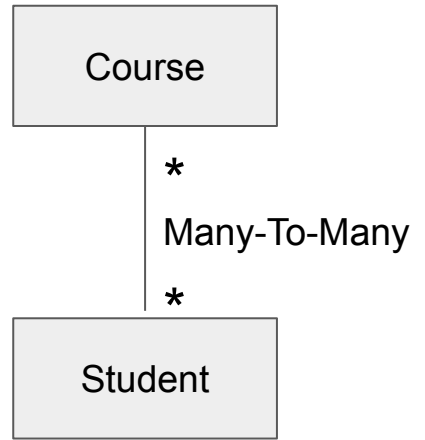
Many-To-Many

- Student follows many Courses
- Course has many Students



Many-To-Many: Database

- Table COURSES
 - Has ID and NAME and ACADEMIC_YEAR
- We use a “crosstable” to link courses to students:
 - STUDENTS_COURSES
 - Has STUDENT_ID: foreign key to STUDENTS
 - Has COURSE_ID: foreign key to COURSES

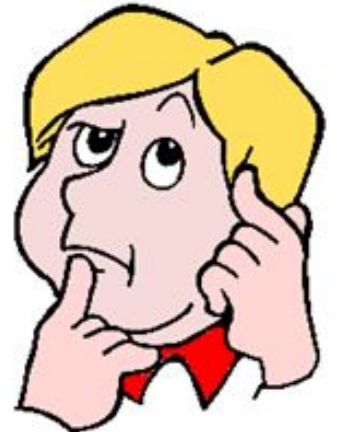
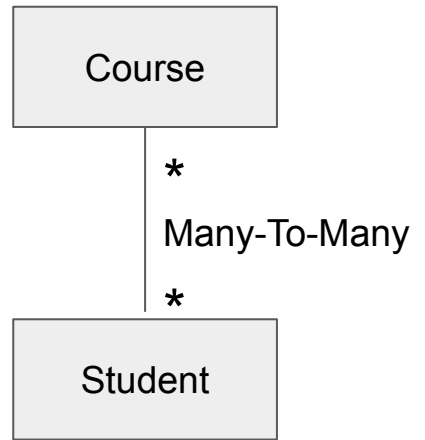


```
create table COURSES
(
    ID                INTEGER auto_increment
        primary key
        unique,
    NAME              CHARACTER VARYING(100) not
null,
    ACADEMIC_YEAR    INTEGER                not null
);
```

```
create table STUDENTS_COURSES
(
    STUDENT_ID INTEGER not null,
    COURSE_ID  INTEGER not null,
    constraint FK_COURSE ID
        foreign key (COURSE_ID) references COURSES,
    constraint FK_STUDENT ID
        foreign key (STUDENT_ID) references
STUDENTS
);
```

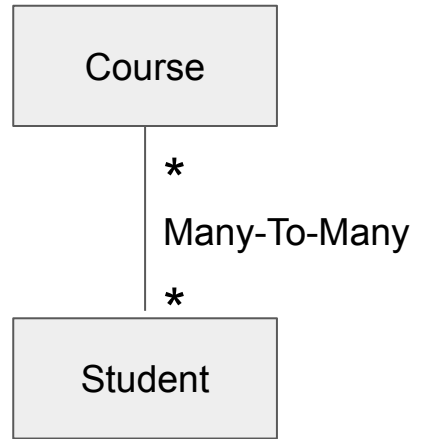
Many-To-Many: in Domain Model

- Course class → is an *entity*
 - Has id, name, academicYear → getters, setters, constructors
 - Has List<Student> students → getter and setter, addStudent
 - toString does not show the students
- Student class:
 - Has List<Course> courses
 - addCourse → also adds student to course
 - Getter - setter
 - We do not add the courses to the toString

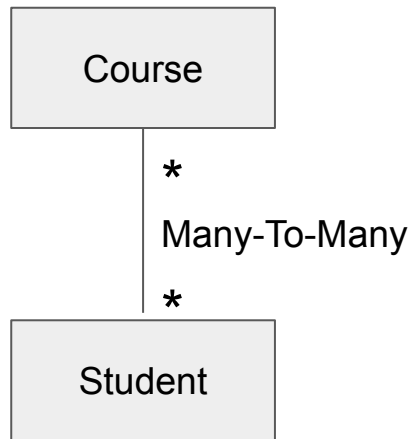


Many-To-Many: let's do it!

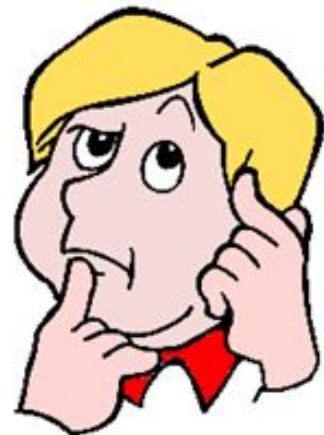
- Database:
 - Add COURSES to filedatabase - add some data
 - Add the crosstable - add some data
 - Generate DDL and SQL INSERTS and update schema.sql and data.sql
- Domain Model
 - Create Course class, see previous slide
 - Update Student class



Many-To-Many: repository?

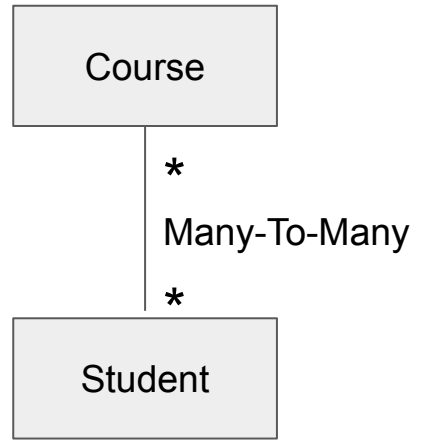


- We need a **CourseRepository**
 - `findAll`, `findById`
 - we do NOT load the Students! (=lazy loading)
 - → that would also load Address and School, for each Student...!
 - `createCourse`, `updateCourse`
 - `deleteCourse`
 - We also need to delete the records in the `STUDENTS_COURSES`!
- **StudentRepository**
 - `findAll`, `findById` → let's load the courses... (eager loading, good idea?)
 - Add a `findByCourse` method
 - `updateStudent`: delete and re-insert the `STUDENTS_COURSES`?
 - `deleteStudent`
 - We also need to delete the records in the `STUDENTS_COURSES`!



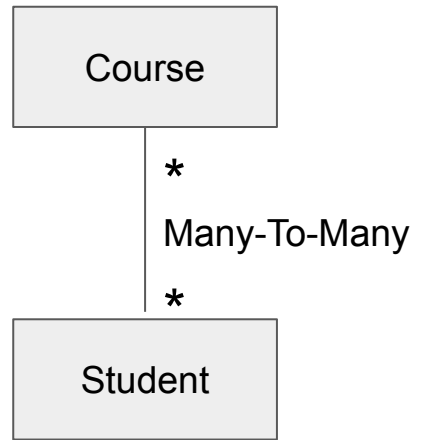
Many-To-Many: let's do it!

- Create a CourseRepository
 - findAll, findById → do NOT load the Students! (=lazy loading)
 - createCourse, updateCourse
 - deleteCourse → also need to delete the records in the STUDENTS_COURSES
- StudentRepository
 - findAll, findById → load the courses...
 - Add a findByCourse method
 - updateStudent: delete and re-insert the STUDENTS_COURSES
 - deleteStudent → delete the records in the STUDENTS_COURSES!

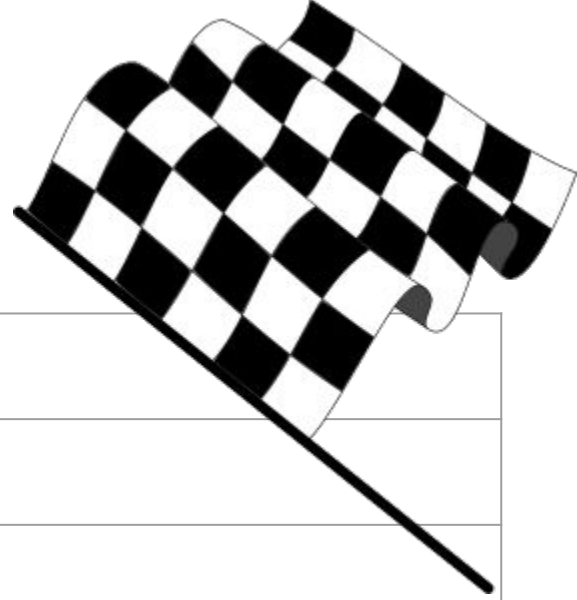


Many-To-Many: StudentMenu

- StudentMenu:
 - Implement “Add course”
 - Implement “Delete course”
 - Implement “Add student to course”
 - Implement “List students of course”
 - Implement “List courses of student”
 - Implement “Remove student from course”
- Test!!!!



Agenda this week



Project review
Implementing relationships in the repository
One-To-One
Many-To-One
Many-To-Many

Project

- Add the relationships to your project
 - Add them in the database, use cross table for many-to-many
 - Adjust/add repository classes where necessary
 - Use the JdbcTemplate implementation
 - When clicking an entity in the table to see its details, you also see a list of its related entities. The items in the list are again clickable.
Example: click on Book shows bookdetails and list of Authors, click on Author shows AuthorDetails and list of Books...
- Add the possibility to delete entities, for example by adding a delete icon at the end of each row of the tables

